

DAY 3 — PYTHON FUNDAMENTALS

For Loop • While Loop • Range • Iteration

MODULE: Python | PRACTICE: 20 loop problems | TASK: Pattern Printing

What You'll Learn Today

- How to repeat actions in code using for and while loops
- Using range() to generate sequences of numbers for loops
- Looping over lists, strings, and other sequences directly
- Controlling loops with break, continue, and pass
- Avoiding common loop mistakes, including infinite loops
- Building a practical Pattern Printing program using nested loops

1 FOR LOOP — Repeating Actions Over a Sequence

What is a For Loop?

A for loop lets your program repeat a block of code once for every item in a sequence — a list, a string, a range of numbers, and so on. Instead of writing the same line 20 times, you write it once and let the loop do the repeating.

Syntax

```
for item in sequence:
    # code runs once for every item
    # 'item' holds the current value each round
```

Example 1 — Looping Over a List

CODE

```
# Looping over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

OUTPUT

```
apple
banana
cherry
```

Example 2 — Looping Over a String

CODE

```
# Strings are sequences too
word = "CODE"
for letter in word:
    print(letter)
```

OUTPUT

```
C
O
D
E
```

Key Points About For Loops

- The colon : after the sequence is required, just like with if
- Code inside the loop must be indented (4 spaces)
- The loop variable (e.g. fruit, letter) can be named anything
- A for loop runs exactly once per item — it stops automatically at the end
- You can loop over lists, strings, tuples, dictionaries, and ranges

2 RANGE() — Generating Number Sequences

range() generates a sequence of numbers without storing them all in memory. It is the most common way to control how many times a for loop runs.

Form	Meaning	Example	Produces
range(stop)	0 up to stop – 1	range(5)	0, 1, 2, 3, 4
range(start, stop)	start up to stop – 1	range(2, 6)	2, 3, 4, 5
range(start, stop, step)	start up to stop, skipping by step	range(0, 10, 2)	0, 2, 4, 6, 8
range(5, 0, -1)	counts backward	range(5, 0, -1)	5, 4, 3, 2, 1

Example — Range Inside a For Loop

CODE

```
# Print numbers 1 to 5 using range
for i in range(1, 6):
    print(i)
```

OUTPUT

```
1
2
3
4
5
```

Important: `range(stop)` never includes the stop value itself — `range(5)` gives 0–4, not 0–5. This is the single most common source of off-by-one mistakes in loops.

3 WHILE LOOP — Repeating Until a Condition Fails

What is a While Loop?

A while loop keeps repeating a block of code as long as its condition stays True. Use it when you don't know in advance exactly how many times you need to repeat — unlike a for loop, which is built for a known sequence.

Syntax

```
while condition:
    # code runs again and again
    # while condition stays True
```

Example 1 — Basic While Loop

CODE

```
# Count from 1 to 5
count = 1
while count <= 5:
    print(count)
    count = count + 1 # without this line, the loop never ends
```

OUTPUT

```
1
2
3
4
5
```

Example 2 — While Loop With User Input

CODE

```
# Keep asking until the user types 'exit'
command = ""
while command != "exit":
    command = input("Type a command ('exit' to quit): ")
    print("You typed:", command)
```

For Loop vs While Loop

For Loop	While Loop
Best when you know how many times to repeat	Best when repetition depends on a changing condition
Iterates directly over a sequence (list, range, string)	Keeps a manual condition variable updated each round
Stops automatically when the sequence ends	Stops only when the condition becomes False

■ **Watch out for infinite loops.** If the condition in a while loop never becomes False, the loop runs forever. Always make sure something inside the loop moves it toward the exit condition (like `count = count + 1` above).

4 LOOP CONTROL — `break`, `continue`, `pass`

These three keywords change how a loop behaves from the inside, and are usually paired with an if condition.

Keyword	Effect
<code>break</code>	Exits the loop immediately, skipping any remaining items
<code>continue</code>	Skips the rest of the current round and moves to the next item
<code>pass</code>	Does nothing — a placeholder so empty code still runs without error

Example — `break` and `continue`

CODE

```
# break stops the loop, continue skips one round
for number in range(1, 10):
    if number == 6:
        break          # stop the loop completely
    if number % 2 == 0:
        continue       # skip even numbers
    print(number)
```

OUTPUT

```
1
3
5
```

5 PRACTICAL EXAMPLE — Pattern Printing

Task: Use nested for loops (a loop inside another loop) to print a triangle of stars, then a number pattern. Pattern printing is one of the best ways to build real intuition for how nested iteration works.

Pattern 1 — Star Triangle

CODE

```
# Star triangle pattern
rows = 5
for i in range(1, rows + 1):
    print("*" * i)
```

OUTPUT

```
*
**
***
****
*****
```

Pattern 2 — Number Triangle (Nested Loop)

CODE

```
# Number triangle using a nested loop
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print() # move to the next line after each row
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Breaking Down the Pattern Logic

- The outer loop (i) controls how many rows get printed
- The inner loop (j) controls how many items appear in each row
- `end=" "` keeps values on the same line instead of a new line each time
- An empty `print()` after the inner loop moves to the next row
- Changing the inner loop's range or values changes the pattern shape

6 COMMON MISTAKES TO AVOID

1. Off-by-One Errors With `range()`

X Wrong	<code>range(1, 5)</code> prints 1, 2, 3, 4
----------------	--

✓ Correct	<code>range(1, 6)</code> to include up to 5
------------------	---

2. Forgetting to Update the While Condition

✗ Wrong	<code>while count <= 5:</code> (no update inside)
✓ Correct	increase or change count inside the loop body

3. Incorrect Indentation

✗ Wrong	loop body not indented under for/while
✓ Correct	always indent with 4 spaces inside the loop

4. Modifying a List While Looping Over It

✗ Wrong	removing items from a list during a for loop over it
✓ Correct	loop over a copy, e.g. <code>for x in list[:]</code>

5. Confusing break and continue

✗ Wrong	using break when you only meant to skip one item
✓ Correct	use continue to skip, break to exit fully

7 QUICK REFERENCE GUIDE

Concept	Syntax	Example
Basic For Loop	<code>for x in seq:</code>	<code>for f in fruits:</code>
Range Loop	<code>for x in range(n):</code>	<code>for i in range(5):</code>
Basic While Loop	<code>while condition:</code>	<code>while count <= 5:</code>
Exit a Loop Early	<code>break</code>	<code>if x == 6: break</code>
Skip One Round	<code>continue</code>	<code>if x % 2 == 0: continue</code>
Nested Loop	<code>for i: for j:</code>	pattern printing

Key Takeaways

- A for loop repeats over a known sequence; a while loop repeats until a condition fails
- `range(start, stop, step)` generates numbers, and never includes the stop value itself
- `break` exits a loop completely; `continue` skips just the current round
- Every while loop needs something inside it that moves the condition toward False
- Nested loops (a loop inside a loop) are the key to printing patterns and grids

- Iteration is the foundation for processing lists, files, and data in every later topic

Next Steps

- Practice: complete the 20 loop practice problems for Day 3
- Build: write the Pattern Printing program (star triangle + number triangle, then try a pyramid)
- Experiment: rewrite one of today's for loops as a while loop, and vice versa
- Upload: save today's loop programs on GitHub

■ **Focus for today: understand iteration.** Don't just get the pattern to print — be able to explain, for every line of code, why the loop runs the number of times it does. That understanding is what makes the next topics (lists, functions, file handling) click faster.